MADA: A Multi-Agent AI System for Autonomous Database Administration

Shauryaveer Singh Dabas — 24BCE0416

BCSE302L Database Systems · Final Review

Abstract

Database administration — indexing, configuration, query tuning, incident diagnosis and security monitoring — is still largely manual, and the AI tools that automate it each cover one duty and act on predictions rather than measurements. This project presents MADA (Multi-Agent Database Administrator), a multi-agent system for autonomous PostgreSQL administration. Six specialist agents (monitor, index, knob, query, diagnosis and security) read shared telemetry, test candidate actions on a sandbox clone of the database, and submit structured proposals to a coordinator that resolves conflicts over shared resources and to a safety gate that admits only changes with measured gain, keeps irreversible actions for human approval and rolls back regressions. On a 168 MB e-commerce database under an eight-session workload, MADA reduced 95th-percentile latency on all ten query templates — the complaints report from 26.4 s to 37 ms and order detail from 223 ms to 2.0 ms — and raised throughput from 60 to 241 statements per second. It detected all seven injected faults within 12.2 s, ranked the correct root cause first for every one and repaired all of them automatically. With the safety gate disabled, all four harmful proposals reached production and post-change latency monitoring rolled back none of them; with the gate enabled, all four were rejected.

Introduction

Almost every modern application depends on a relational database, and that database stays fast and safe only through continuous expert work: choosing indexes, setting memory and planner parameters, rewriting slow SQL, diagnosing incidents and watching for misuse. Pavlo et al. argued that a database management system should take on much of this work itself — a “self-driving” system that forecasts its own workload and plans its own changes [1]. In the years since, research on AI for databases has produced learned components for nearly every one of these duties [2]: knob tuners that learn configurations from earlier tuning sessions [3], cloud services that index millions of databases automatically [4], learned optimisers that steer query plans with hints [6], and diagnosis tools that explain anomalies from performance metrics [7].

These advances arrive as separate tools, each owning one duty and each trusting its own predictions, and two problems follow. First, the duties interact. A new index changes query plans, memory use and the cost of every write, so a knob tuner, an index advisor and a query rewriter running side by side can undo one another's work. Second, a prediction is not an outcome: Ding et al. showed that optimiser cost estimates frequently recommend indexes that make real queries slower [5]. Security sits further apart still — detectors that profile each role's normal access [9] usually run outside the tuning loop, so nothing decides whether a performance change or a security response should come first.

Large language models add a new option and a new risk. An LLM agent can read documentation and diagnose incidents [8], and several agents can divide a task between them through conversation [10]. But an agent that holds database credentials can also run a wrong statement directly on production, and research prototypes rarely put a verified safety layer between an agent's proposal and the live system.

This project builds and evaluates **MADA (Multi-Agent Database Administrator)**. Six specialist agents — monitor, index, knob, query, diagnosis and security — share one view of a PostgreSQL database,

test every idea on a sandbox clone, and send structured proposals to a coordinator and a safety gate. The coordinator resolves conflicts over shared resources and orders actions by priority (security, availability, correctness, performance). The gate applies only changes backed by measured evidence, keeps irreversible actions for a human, and rolls back any change that makes the system slower. A working prototype was evaluated on a 168 MB e-commerce database under a live eight-session workload, with seven injected faults and an ablation study of the safety gate.

Literature Review

Self-driving databases and configuration tuning. The self-driving DBMS vision [1] relies on forecasting what the workload will do next; QB5000 clusters queries into templates and forecasts their arrival rates so that indexes can be built before demand arrives [11]. Configuration tuning has moved from Gaussian-process models that reuse past tuning data (OtterTune [3]) to deep reinforcement learning (CDBTune [12], QTune [13]) and, most recently, to language models that read the manual and narrow the search before Bayesian optimisation (DB-BERT [14], GPTuner [15]). Each of these tunes configuration knobs only, and the learning-based ones need many trial runs on the instance being tuned.

Index selection. Index advisors date back to AutoAdmin's what-if analysis, in which the optimiser costs hypothetical indexes without building them [16]. Later work improved the search — Extend's recursive multi-attribute strategy [17] and deep-reinforcement-learning advisors [19] — and a broad experimental evaluation compared eight algorithms under one framework [18]. Two papers question the cost estimates themselves: Ding et al. show that estimated benefit often disagrees with measured execution [5], and Microsoft's Azure SQL Database validates every automatic index change after deployment and reverts regressions [4].

Query optimisation and rewriting. Neo learned a whole query optimiser from experience [20]; Bao made the approach practical by choosing planner hint sets for an existing optimiser [6]. Query rewriting has been framed as a search over rule applications with Monte Carlo tree search [21] and, with LLM-R2, as an LLM choosing rules from a rule engine so that every rewrite stays valid [22].

Diagnosis and security. DBSherlock explains performance anomalies with predicates that separate the anomaly from normal operation [7]; iSQUAD diagnoses intermittent slow queries in a cloud fleet by matching incidents to known cases [23]; D-Bot uses several collaborating LLMs to reason from documentation and metrics to a root cause [8]. On the security side, Kamra et al. profile which data each role normally touches and flag queries that do not fit [9], and Halfond et al. classify SQL-injection attacks and countermeasures [24].

Agents. ReAct interleaves reasoning with tool calls [25], and AutoGen lets several LLM agents cooperate through conversation [10]. These frameworks supply the agent loop but say nothing about database-specific safety: which statements an agent may run, how its claims are checked, or when a person must approve.

Below are the reference articles that are closest to this topic but differ in implementation.

Table 1. Related work

Ref. No.	Year of publication	Authors and Title	Contributions	Limitations
1	2017	Pavlo et al. Self-Driving Database Management Systems.	1. Architecture for a DBMS that forecasts its workload and plans its own tuning actions. 2. Peloton prototype as a research platform for autonomous operation.	1. Vision paper with an early prototype; no end-to-end evaluation of actions. 2. Tied to a custom in-memory DBMS rather than an existing engine.
3	2017	Van Aken, Pavlo, Gordon and Zhang. Automatic DBMS Tuning Through Large-scale Machine Learning (OtterTune).	1. Reuses data from earlier tuning sessions by mapping a new workload to a known one. 2. Ranks important knobs with Lasso and recommends settings with Gaussian-process regression.	1. Configuration knobs only; no indexes, queries or incidents. 2. Needs a large repository of past observations and many test runs.
11	2018	Ma et al. Query-based Workload Forecasting for Self-Driving DBMSs (QB5000).	1. Clusters queries into templates and forecasts arrival rates per cluster. 2. Uses forecasts to choose indexes ahead of demand.	1. Forecasting only; acting on the forecast is left to other components. 2. Ensemble models add training overhead for small deployments.
12	2019	Zhang et al. An End-to-End Automatic Cloud Database Tuning System Using Deep RL (CDBTune).	1. Deep deterministic policy gradient learns knob settings by trial and error. 2. Reward built from throughput and latency changes; no labelled data needed.	1. Many training iterations, each running the workload. 2. Exploration on a live instance can apply harmful settings.
13	2019	Li, Zhou, Li and Gao. QTune: A Query-Aware Database Tuning System with Deep RL.	1. Featurises SQL queries so tuning reacts to the actual workload. 2. Supports query-, workload- and cluster-level tuning.	1. Knob tuning only. 2. Costly training before useful recommendations.
14	2022	Trummer. DB-BERT: A Database Tuning Tool that Reads the Manual.	1. Extracts tuning hints from manuals and web text with a language model. 2. Uses reinforcement learning to decide which hints to apply.	1. Quality depends on the text available for the DBMS. 2. Configuration only; changes tried directly on the system.
15	2024	Lao et al. GPTuner: Manual-Reading Database Tuning via GPT-Guided Bayesian Optimization.	1. LLM turns manuals and forum text into structured knob knowledge and narrowed ranges. 2. Coarse-to-fine Bayesian optimisation finds good settings in few trials.	1. LLM calls add cost and latency. 2. Covers knobs only; no coordination with indexing or query tuning.

Ref. No.	Year of publication	Authors and Title	Contributions	Limitations
16	1997	Chaudhuri and Narasayya. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server.	1. What-if analysis: the optimiser costs hypothetical indexes without building them. 2. Per-query candidate selection and greedy enumeration for the whole workload.	1. Relies on optimiser cost estimates, which can be wrong. 2. Offline tool; a DBA still decides and applies.
17	2019	Schlosser, Kossmann and Boissier. Efficient Scalable Multi-Attribute Index Selection Using Recursive Strategies (Extend).	1. Recursive greedy extension builds multi-column indexes step by step. 2. Scales to large workloads with good quality under a storage budget.	1. Quality bounded by the cost model. 2. Index selection only.
18	2020	Kossmann, Halfpap, Jankrift and Schlosser. An Experimental Evaluation of Index Selection Algorithms.	1. Compares eight index selection algorithms on TPC-H, TPC-DS and JOB. 2. Open-source evaluation platform with what-if cost support.	1. Evaluation uses estimated costs, not measured run times. 2. No guidance on validating choices before deployment.
19	2020	Lan, Bao and Peng. An Index Advisor Using Deep Reinforcement Learning.	1. Deep Q-network chooses indexes from heuristically generated candidates. 2. Handles multi-attribute indexes and index interactions.	1. Needs training per workload. 2. Reward based on cost estimates.
4	2019	Das et al. Automatically Indexing Millions of Databases in Microsoft Azure SQL Database.	1. Production service that recommends, creates and drops indexes across millions of databases. 2. Validates each change after deployment and reverts regressions.	1. Index management only. 2. Validation happens on production, after the change.
5	2019	Ding et al. AI Meets AI: Leveraging Query Executions to Improve Index Recommendations.	1. Shows optimiser cost estimates often predict index benefit wrongly. 2. Classifier trained on execution data cuts query regressions from index changes.	1. Needs execution history for training. 2. Addresses index tuning only.
20	2019	Marcus et al. Neo: A Learned Query Optimizer.	1. End-to-end learned optimiser using a value network over plan trees. 2. Bootstraps from an existing optimiser and improves with experience.	1. Long training time. 2. Sensitive to schema and workload change.
6	2021	Marcus et al. Bao: Making Learned Query Optimization Practical.	1. Chooses planner hint sets per query with Thompson sampling over a tree model. 2. Runs on top of PostgreSQL with modest training.	1. Limited to the hint sets supplied. 2. Does not rewrite SQL, change indexes or configuration.

Ref. No.	Year of publication	Authors and Title	Contributions	Limitations
21	2021	Zhou, Li, Chai and Feng. A Learned Query Rewrite System using Monte Carlo Tree Search.	1. Searches the order of rewrite-rule application with MCTS. 2. Learned estimator predicts cost reduction of a rewrite.	1. Bounded by the rule set. 2. Relies on estimated rather than measured benefit.
22	2024	Li, Yuan, Wang, Cong and Bing. LLM-R2: An LLM-Enhanced Rule-based Rewrite System.	1. LLM selects rewrite rules for a rule engine, so every rewrite stays valid. 2. Contrastive demonstration selection improves the LLM's choices.	1. LLM inference cost per query. 2. Rewrite only; no link to indexing or monitoring.
7	2016	Yoon, Niu and Mozafari. DBSherlock: A Performance Diagnostic Tool for Transactional Databases.	1. Generates predicates that separate an anomaly region from normal operation. 2. Builds causal models of root causes from user feedback.	1. User must mark the anomaly region. 2. Explains causes but does not apply fixes.
23	2020	Ma et al. Diagnosing Root Causes of Intermittent Slow Queries in Cloud Databases (iSQUAD).	1. Anomaly extraction and dependency cleansing on KPIs for slow queries. 2. Bayesian case-based clustering matches new incidents to known causes.	1. Needs labelled historical cases. 2. Diagnosis only.
8	2024	Zhou et al. D-Bot: Database Diagnosis System using Large Language Models.	1. LLM extracts diagnosis knowledge from documents and reasons over metrics with tree search. 2. Multiple LLMs collaborate to produce a root-cause report.	1. LLM cost, latency and risk of unsupported conclusions. 2. Suggested fixes are not tested before use.
9	2008	Kamra, Terzi and Bertino. Detecting Anomalous Access Patterns in Relational Databases.	1. Learns per-role profiles of SQL access from query logs. 2. Flags queries whose access pattern does not fit the issuing role.	1. Assumes role-based access control and clean training logs. 2. Detection only; response left to administrators.
24	2006	Halfond, Viegas and Orso. A Classification of SQL-Injection Attacks and Countermeasures.	1. Taxonomy of injection mechanisms and attack types. 2. Compares detection and prevention techniques against that taxonomy.	1. Survey; no new detector is evaluated. 2. Predates learned detectors and automated response.

Gaps Identified

- **One duty per tool.** Every system in Table 1 handles a single duty — knobs, indexes, queries, diagnosis or security. None coordinates them, although an index changes plans, memory and write cost at once.
- **Predictions trusted over measurements.** Most tuners act on optimiser cost estimates or model predictions. Execution-based validation [4], [5] exists for indexes only, and in [4] it happens after the change is already live.

- **No safety layer for agent actions.** LLM-based tools [8], [15], [22] can propose SQL or settings, but none checks proposals against an allow-list, demands measured evidence, separates reversible from irreversible actions, or requires human approval for the latter.
- **Security separated from performance.** Access-anomaly and injection detection [9], [24] run apart from tuning, so no component decides which action takes priority when they compete.
- **Evaluation in isolation.** Systems are tested on one task at a time; few are evaluated end to end on a live workload with injected faults and an ablation of their safety mechanisms.

Proposed Method

MADA separates *thinking* from *acting*. The agents never change production: they read telemetry, test ideas on a sandbox clone, and emit structured proposals — action SQL, rollback SQL, resources touched, risk level and evidence. Only the coordinator and safety gate apply changes. Figure 1 shows the architecture.

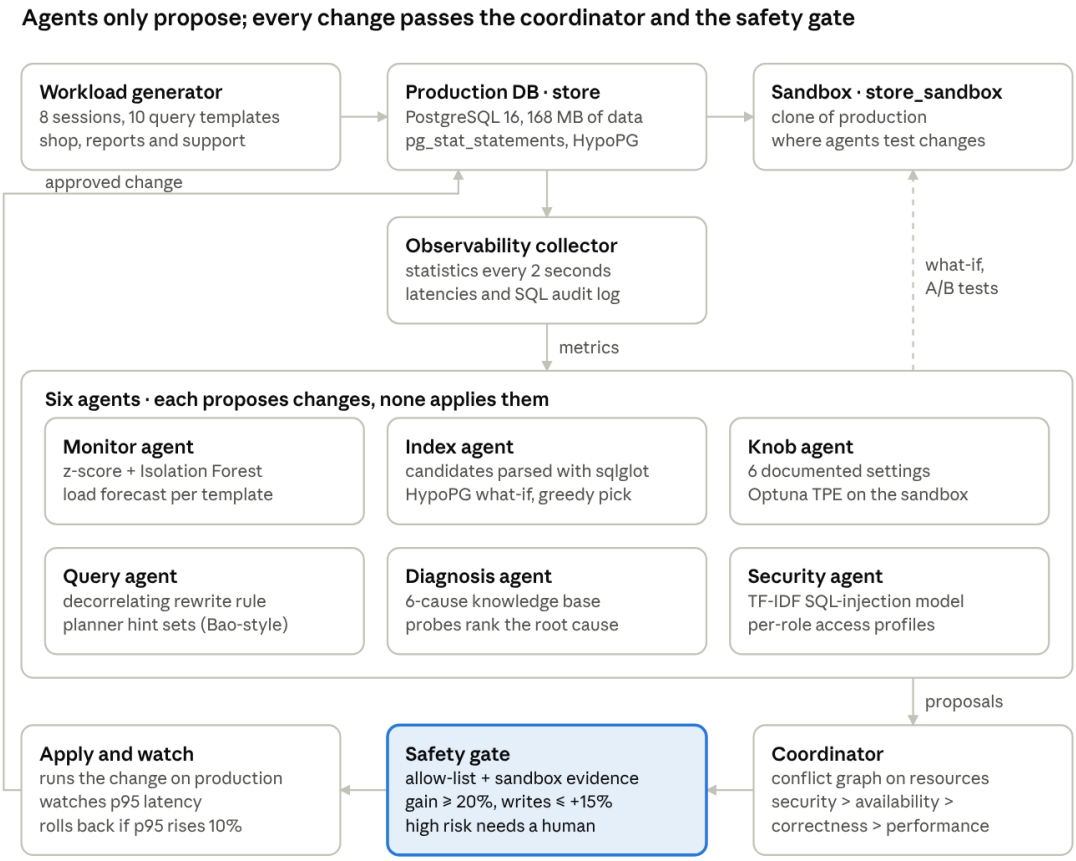


Figure 1. MADA architecture: telemetry in, proposals out, one gated path to production

Components

Testbed and workload generator. Builds an e-commerce schema of six tables (customers, products, orders, order_items, reviews, payments) with primary keys only, loads seeded data (168 MB), and clones it into a sandbox with CREATE DATABASE … TEMPLATE. Eight threads replay ten parameterised query templates under three roles (application, analyst, support). Every statement's latency and SQL go to a recorder that doubles as the audit log. Before each statement the workload reads two registry tables, mada.hints and mada.rewrites — the only way an agent can change the SQL the application runs.

Observability collector and monitor agent. Every 2 s the collector reads pg_stat_database, pg_stat_activity, pg_locks and pg_stat_user_tables, turns counters into per-second rates over a 10 s window, and adds host CPU and the 95th-percentile (p95) latency of each template. The monitor learns a baseline (mean and spread per metric plus an Isolation Forest [26] over all metrics) and raises an alert when a metric is more than 4σ from normal for two consecutive ticks and the forest agrees, or more than 12σ on its own. It also forecasts each template's arrival rate a minute ahead, following QB5000 [11].

Query agent. Takes analytical templates whose mean latency exceeds 250 ms. It tries rule-based rewrites first — rule R1 parses the SQL with sqlglot and turns a correlated COUNT(*) subquery into a LEFT JOIN on a grouped derived table, then checks that the rewrite returns identical rows for sample parameters — and then times Bao-style planner hint sets [6] on the sandbox. Only a variant at least 20% faster becomes a proposal.

Index agent. Reads the ten most expensive statements from pg_stat_statements, parses them with sqlglot to collect candidate one- and two-column indexes, and costs each candidate as a hypothetical index with HypoPG and EXPLAIN (GENERIC_PLAN) [16]. A greedy search in the style of Extend [17] keeps the candidates with the largest frequency-weighted cost reduction within a 120 MB budget. Because estimated cost is not run time [5], each pick is then built for real on the sandbox and the affected templates and the write path are timed with and without it.

Knob agent. Tunes six reload-only settings (work_mem, hash_mem_multiplier, random_page_cost, effective_cache_size, max_parallel_workers_per_gather, jit). A catalog written from the PostgreSQL manual gives each setting's meaning and safe range, as in GPTuner [15]; Optuna's TPE sampler [27], [28] runs 14 trials, each replaying the current workload mix on the sandbox weighted by observed frequency.

Diagnosis agent. Finds DBSherlock-style predicates [7] — metrics whose anomaly-window mean is at least 3σ from normal — and matches them against a knowledge base of six root causes (lock contention, work_mem spill, missing index, table bloat, connection storm, CPU saturation). Each cause has a metric signature and a confirming SQL probe; for lock contention the probe follows pg_blocking_pids() to the blocking session. The anomaly is compared with the minutes just before the fault, so leftovers of an earlier incident do not look new; a cause whose probe confirms it always ranks above an unconfirmed one, and only a confirmed cause gets an automatic fix, routed to the agent that owns it.

Security agent. A TF-IDF model over character 2- to 5-grams with logistic regression, trained on 1,500 benign search statements and 700 injected ones from five attack families [24], screens every statement from the product search — the one place user input is spliced into SQL. An access monitor learns which tables each role touches in normal operation [9] and flags reads outside that profile; the response revokes the grant and ends the role's sessions.

Coordinator and safety gate. Every proposal declares the resources it touches (a table, the planner, memory, a session or a role). The coordinator orders proposals by priority (security > availability > correctness > performance), then by gain and risk, and defers any proposal whose resources are already claimed in the current window. The safety gate then checks the rules in Table 2. After applying a change, the coordinator watches the p95 latency of the affected templates and rolls the change back if it rises by more than 10%.

Table 2. Safety-gate rules, checked in order

Check	Rule
Static policy	Every statement matches an allow-list of statement shapes (CREATE/DROP INDEX, ALTER SYSTEM for catalogued knobs, registry inserts, VACUUM, GRANT/REVOKE, pg_terminate_backend); settings lie in their documented ranges; reversible actions carry a rollback statement.
Sandbox evidence	Performance proposals show at least 20% measured gain on the sandbox and at most 15% extra write cost; rewrites return identical rows on sample parameters.
Risk	Low and medium risk apply automatically; high-risk, irreversible actions (ending sessions, revoking access) need human approval.
Watch and rollback	After apply, p95 of the affected templates is watched; a rise of more than 10% triggers the rollback statement.

Workflow

Tuning cycle. (1) The workload runs and the collector records two minutes of baseline. (2) The query agent fixes slow statements first, so later agents tune the SQL that production will actually run. (3) The index agent proposes indexes from a fresh minute of statistics. (4) The knob agent searches configurations against the tuned workload. (5) After each agent, the coordinator runs one or more windows: order, defer conflicts, gate, apply, watch, roll back if needed.

Incident loop. Every 2 s the monitor checks the latest metrics and the security agent scans new audit entries. A security finding goes straight to the coordinator with top priority. A performance alert triggers the diagnosis agent, which ranks causes, confirms the top one with its probe and proposes the owning agent's fix; the coordinator and gate then decide as above.

Implementation

The prototype is about 2,800 lines of Python and JavaScript, running against PostgreSQL 16.15 with pg_stat_statements and HypoPG 1.4 on one Ubuntu 24.04 machine with 2 vCPUs and 7 GB RAM. It uses psycopg 3, sqlglot, scikit-learn and Optuna. A live web dashboard (Flask with Chart.js) shows the agents' log, every coordinator and safety-gate decision and each template's latency, and lets an operator run the tuning cycle, inject faults, approve or reject high-risk actions and switch the gate off; the whole system starts with one docker compose command. LLM support is optional; all results reported here were produced in rule mode, with explanations taken from the knowledge base, so none depends on an LLM. In the experiment script, high-risk approvals were given automatically and are labelled as such in the logs.

Result in brief

After one tuning cycle, p95 latency fell on all ten query templates and throughput rose 4×; four of six proposals passed the gate. In the fault phase MADA detected all seven injected faults within 12.2 s and fixed all of them. The next section gives the full experimental analysis.

Experimental Analysis

Setup

All results come from one scripted run of the final prototype (run_experiments.py). The testbed is the 168 MB e-commerce database described above, with primary-key indexes and PostgreSQL defaults at the start. Eight sessions run continuously: five application sessions (order history, order detail, product

reviews, product lookup, product search, placing an order), two analyst sessions (revenue by category, top customers per city, products with the most complaints) and one support session. Measurement windows are two minutes. Metrics are p95 latency per template, throughput, time to detect and to fix each fault, rank of the true root cause, classifier precision and recall, and the outcome of each harmful proposal with and without the gate.

Tuning results

Table 3 lists every proposal from the tuning cycle and the gate's decision: four were applied and two rejected on their sandbox evidence. Figure 2 shows the query agent at work and Figure 3 the resulting latencies.

Table 3. Proposals in the tuning cycle

#	Proposal (agent)	Decision	Evidence
1	Rewrite R1 of the complaints report (query)	Applied	24.3 s → 88 ms on the sandbox; identical rows
2	Index orders(order_date) (index)	Rejected	support lookup 74 ms → 680 ms; order history 48 ms → 72 ms on the sandbox (measured gain on sandbox -517% < 20%)
3	Index order_items(order_id) (index)	Applied	Production p95 of order detail 191 ms → 9.9 ms
4	Index orders(customer_id, order_date) (index)	Applied	Production p95 of support lookup and order history 89 ms → 2.1 ms
5	Index reviews(product_id, rating) (index)	Rejected	complaints report 87 ms → 80 ms on the sandbox (measured gain on sandbox 8% < 20%)
6	Index reviews(product_id, review_date) (index)	Applied	Production p95 of product reviews 73 ms → 1.3 ms

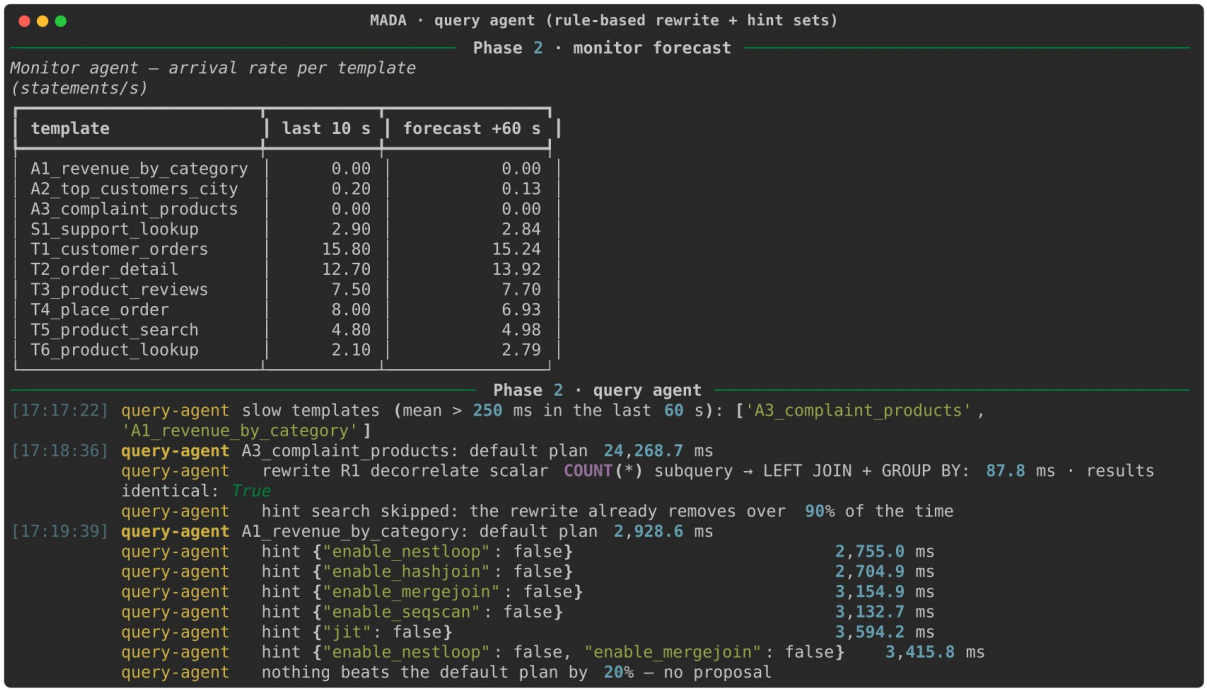


Figure 2. Monitor forecast and query agent: rewrite R1 cuts the complaints report from 24.3 s to 88 ms with identical results; no hint set beats the revenue report's default plan by 20%

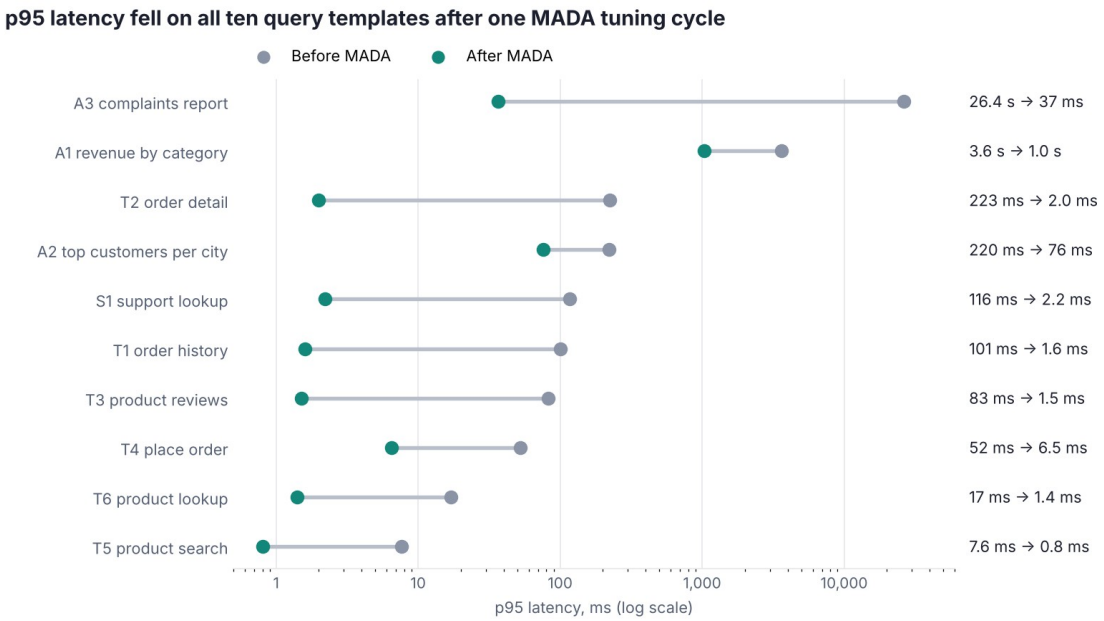


Figure 3. p95 latency per query template before and after the tuning cycle (log scale)

Throughput rose from 60.4 to 240.5 statements per second (4×) with the same eight sessions, because each session waits less per statement. The revenue and city reports were not changed directly, yet they also became faster (-71% and -65%); the most likely reason is that order lookups stopped scanning whole tables, which freed the two CPUs that every session shares.

Fault injection

Seven faults were injected one at a time into the tuned system (Table 4). “Fixed after” is the time from injection until the fix finished on production.

Table 4. Fault-injection results

Fault	Detected after	Cause ranked first	Fix	Fixed after
F1 Index dropped	4.9 s	Missing index (correct)	Index recreated	6.2 s
F2 work_mem set to 64 kB	12.2 s	work_mem spill (correct)	work_mem restored to 4MB	12.2 s
F4 Long transaction holding row locks	6.1 s	Lock contention (correct)	Blocking session ended (approved)	6.1 s
F5 Table bloat, autovacuum off	8.2 s	Table bloat (correct)	VACUUM run, autovacuum re-enabled	8.7 s
F8 80 idle connections	4.1 s	Connection storm (correct)	84 idle sessions ended (approved)	4.1 s
F9 SQL injection on search	2 s	SQL injection (correct)	Attacker session ended (approved)	2 s
F10 Support reads card data	2 s	Abnormal access (correct)	Grant revoked, sessions ended (approved)	2 s

All seven faults were detected within 12.2 s, all seven root causes were ranked first, and every fault was fixed without manual commands. Two refinements made during testing made this possible: the diagnosis agent now compares the anomaly with the minutes just before the fault, so leftovers of an

earlier incident no longer look new, and a cause confirmed by its SQL probe always ranks above an unconfirmed one.

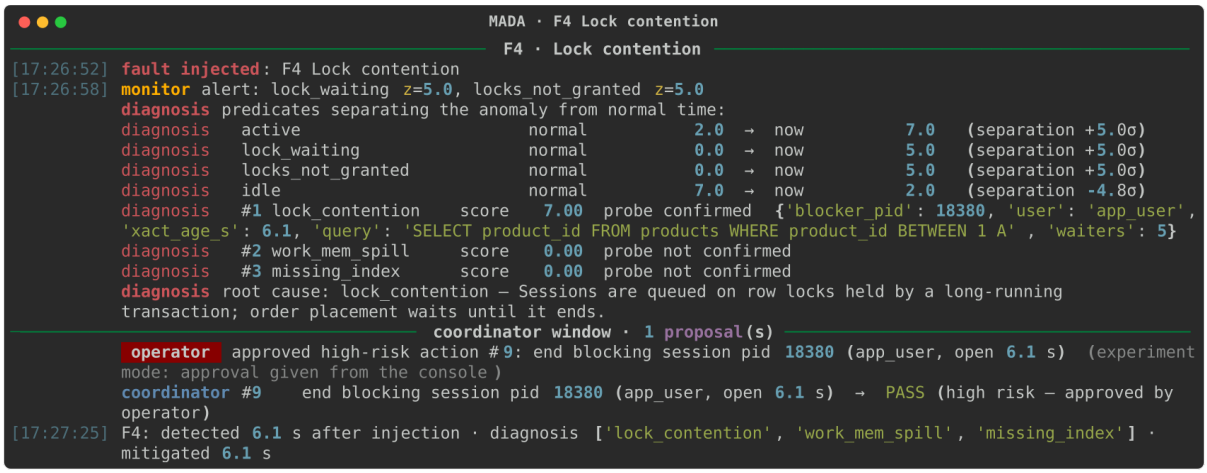


Figure 4. Fault F4: alert 6.1 s after injection; predicates and the probe identify the blocking session, which is ended after approval

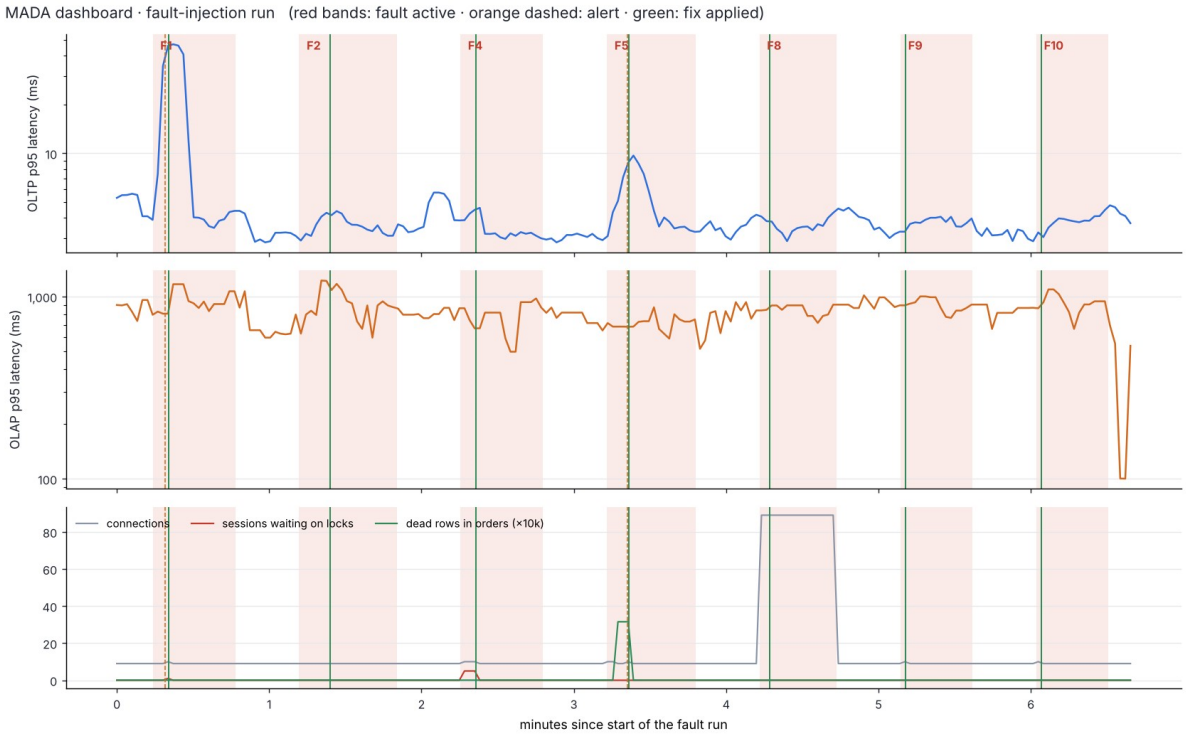


Figure 5. Dashboard of the fault run: red bands mark active faults, orange dashed lines alerts, green lines applied fixes

Security

The injection classifier reached precision 1.000 and recall 1.000 on 660 held-out statements (442 benign, 218 injected). Trained without one attack family (time-based injection) and tested on it, precision was 1.000 and recall 0.793 — the more realistic estimate for attacks not seen in training. Both security faults were caught within 2 s of the first malicious statement.

Safety-gate ablation

Four harmful proposals were sent through the coordinator twice, once with the gate on and once with it off (Table 5).

Table 5. Harmful proposals with and without the safety gate

Harmful proposal	Gate on	Gate off
Disable hash joins for the revenue report	Rejected: 4% slower on the sandbox	Applied and kept
work_mem 1 MB to “save memory”	Rejected: 11% slower on the sandbox	Applied and kept
Faulty rewrite that drops the rating filter	Rejected: results differ	Applied and kept; wrong report served
random_page_cost = 50	Rejected: outside the documented 1.1–4.0 range	Applied and kept

With the gate off, all four harmful changes reached production and the post-change latency watch rolled back none of them. The faulty rewrite was as fast as the correct one, and short latency windows on a busy two-CPU host are noisy. Measured evidence before a change, not monitoring after it, is what protected production.

Comparison with other research works

Table 6 compares MADA's scope with representative systems from the literature. It compares what each system covers, not reported numbers, because the systems were evaluated on different databases and workloads.

Table 6. Scope compared with related systems

System	Knobs	Indexes	Queries	Diagnosis	Security	Tested before going live	Unified control of duties
OtterTune [3]	Yes	No	No	No	No	No	No
CDBTune [12]	Yes	No	No	No	No	No	No
GPTuner [15]	Yes	No	No	No	No	No	No
Azure auto-indexing [4]	No	Yes	No	No	No	After deploy, with revert	No
AI Meets AI [5]	No	Yes	No	No	No	Predicted from past runs	No
Bao [6]	No	No	Hints	No	No	No	No
LLM-R2 [22]	No	No	Rewrites	No	No	No	No
DBSherlock [7]	No	No	No	Yes	No	—	No
D-Bot [8]	No	No	No	Yes (LLM)	No	No	No
Kamra et al. [9]	No	No	No	No	Yes	—	No
MADA (this work)	Yes	Yes	Rewrites + hints	Yes (rules)	Yes	Yes, on a sandbox	Yes

- **Index estimates versus execution.** MADA's run reproduces the central finding of Ding et al. [5]: HypoPG's cost model picked two indexes that real execution did not justify — orders(order_date) slowed the support lookup from 74 ms to 680 ms on the sandbox, and reviews(product_id, rating) saved only 8%. Unlike Azure's after-deployment validation [4], MADA measures on a sandbox before the change, and the ablation shows why that matters.
- **Configuration tuning.** OtterTune, CDBTune and GPTuner [3], [12], [15] report substantial gains from knob tuning on large workloads. On this 168 MB database, which fits in memory, no configuration beat the PostgreSQL defaults, so the knob agent proposed nothing — two replays of the very same default configuration took 0.63 s and 0.45 s, which shows how noisy short measurements are on a shared two-CPU host and why the gate demands at least 20%. Knob tuning is expected to matter more when data exceeds memory; that case was not tested.
- **Hints and rewrites.** As in Bao [6], hint sets helped some queries and not others: none improved the revenue report by 20%. The rule-based rewrite, in the spirit of LLM-R2 [22] but without an LLM, gave the single largest gain (24.3 s → 88 ms) and was verified for identical results before use.
- **Diagnosis.** D-Bot [8] reasons with LLMs over documentation; MADA's rule-based knowledge base with SQL probes ranked the true cause first in all seven faults, but cannot explain a cause outside its six entries.
- **Limitations of this evaluation.** Each experiment ran once on a shared 2-vCPU machine, so short latency measurements are noisy; the LLM path was not evaluated; and high-risk approvals were automatic in the experiment script (the live dashboard asks the operator).

Future Work

- Add an LLM planner, in the style of D-Bot [8], for incidents outside the six-cause knowledge base — behind the same safety gate.
- Use workload forecasts [11] to schedule index builds and vacuum before peaks rather than after alerts.
- Add a recovery agent for replicas, failover and verified backups.
- Repeat the evaluation several times on larger datasets that exceed memory, where configuration tuning should matter more, and report confidence intervals.
- Feed gate decisions and rollbacks back to the agents as training data so they learn which proposals fail.
- Extend the security agent from the search endpoint to every entry point, and add parameterised-query rewriting so injection is removed at its source.

Conclusion

This project designed, built and evaluated MADA, a multi-agent system that administers a PostgreSQL database end to end. Its central idea is that agents may think freely but act only through a coordinator and a safety gate that demand measured evidence, keep irreversible actions for people and roll back regressions. In the final measured run, MADA cut p95 latency on all ten query templates, raised throughput 4×, detected all seven injected faults within 12.2 s, named the right cause first for every one and fixed each on its own. Without the gate, all four harmful changes reached production unnoticed; with it, none did. The working prototype and its live dashboard show that coordinated agents behind a verified safety layer are a practical path towards a database that looks after itself.

References

1. Pavlo, Andrew, et al. "Self-Driving Database Management Systems." *Conference on Innovative Data Systems Research (CIDR)* (2017).
2. Li, Guoliang, et al. "AI Meets Database: AI4DB and DB4AI." *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)* (2021).
3. Van Aken, Dana, et al. "Automatic Database Management System Tuning Through Large-Scale Machine Learning." *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)* (2017): 1009-1024.
4. Das, Sudipto, et al. "Automatically Indexing Millions of Databases in Microsoft Azure SQL Database." *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)* (2019): 666-679.
5. Ding, Bailu, et al. "AI Meets AI: Leveraging Query Executions to Improve Index Recommendations." *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)* (2019): 1241-1258.
6. Marcus, Ryan, et al. "Bao: Making Learned Query Optimization Practical." *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)* (2021): 1275-1288.
7. Yoon, Dong Young, et al. "DBSherlock: A Performance Diagnostic Tool for Transactional Databases." *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)* (2016): 1599-1614.
8. Zhou, Xuanhe, et al. "D-Bot: Database Diagnosis System using Large Language Models." *Proceedings of the VLDB Endowment* 17.10 (2024).
9. Kamra, Ashish, et al. "Detecting Anomalous Access Patterns in Relational Databases." *The VLDB Journal* 17.5 (2008): 1063-1077.
10. Wu, Qingyun, et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv preprint arXiv:2308.08155* (2023).
11. Ma, Lin, et al. "Query-Based Workload Forecasting for Self-Driving Database Management Systems." *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)* (2018): 631-645.
12. Zhang, Ji, et al. "An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning." *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)* (2019): 415-432.
13. Li, Guoliang, et al. "QTune: A Query-Aware Database Tuning System with Deep Reinforcement Learning." *Proceedings of the VLDB Endowment* 12.12 (2019): 2118-2130.
14. Trummer, Immanuel. "DB-BERT: A Database Tuning Tool that 'Reads the Manual'." *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)* (2022).
15. Lao, Jiale, et al. "GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization." *Proceedings of the VLDB Endowment* 17.8 (2024).
16. Chaudhuri, Surajit, and Vivek Narasayya. "An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server." *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)* (1997): 146-155.
17. Schlosser, Rainer, et al. "Efficient Scalable Multi-Attribute Index Selection Using Recursive Strategies." *2019 IEEE 35th International Conference on Data Engineering (ICDE)* (2019): 1238-1249.

18. Kossmann, Jan, et al. "Magic Mirror in My Hand, Which Is the Best in the Land? An Experimental Evaluation of Index Selection Algorithms." *Proceedings of the VLDB Endowment* 13.11 (2020): 2382-2395.
19. Lan, Hai, et al. "An Index Advisor Using Deep Reinforcement Learning." *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)* (2020): 2105-2108.
20. Marcus, Ryan, et al. "Neo: A Learned Query Optimizer." *Proceedings of the VLDB Endowment* 12.11 (2019): 1705-1718.
21. Zhou, Xuanhe, et al. "A Learned Query Rewrite System using Monte Carlo Tree Search." *Proceedings of the VLDB Endowment* 15.1 (2021): 46-58.
22. Li, Zhaodonghui, et al. "LLM-R2: A Large Language Model Enhanced Rule-Based Rewrite System for Boosting Query Efficiency." *Proceedings of the VLDB Endowment* 18.1 (2024).
23. Ma, Minghua, et al. "Diagnosing Root Causes of Intermittent Slow Queries in Cloud Databases." *Proceedings of the VLDB Endowment* 13.8 (2020): 1176-1189.
24. Halfond, William G. J., et al. "A Classification of SQL-Injection Attacks and Countermeasures." *Proceedings of the IEEE International Symposium on Secure Software Engineering (ISSSE)* (2006).
25. Yao, Shunyu, et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)* (2023).
26. Liu, Fei Tony, et al. "Isolation Forest." *2008 Eighth IEEE International Conference on Data Mining (ICDM)* (2008): 413-422.
27. Akiba, Takuya, et al. "Optuna: A Next-Generation Hyperparameter Optimization Framework." *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2019): 2623-2631.
28. Bergstra, James, et al. "Algorithms for Hyper-Parameter Optimization." *Advances in Neural Information Processing Systems* 24 (2011).